Eğer **Emacs benzeri bir Lisp motoru** istiyorsan en mantıklı yaklaşım:

Seçenek 1: Guile'ı gömülü Lisp motoru olarak kullanmak

Bu yaklaşımda Lisp yorumlayıcısını sen yazmazsın.

Mimarin şöyle olur:

UMFS Core (C/C++)
    |
    ▼
Guile Runtime
    |
    ▼
Kullanıcı Scriptleri (.scm)

Örneğin C tarafında:

```
#include <libguile.h>

void create_volume(char *name)
{
   printf("Volume created: %s\n", name);
}
```

Bu fonksiyonu Guile'a export edersin:

```
scm_c_define_gsubr(
   "create-volume",
   1, 0, 0,
   scm_create_volume
);
```

Sonra kullanıcı:

```
(create-volume "data01")
```

yazar.

GNU Guix, GDB ve LilyPond buna benzer şekilde Guile kullanıyor.

Avantajları:

- Çöp toplayıcı hazır
- REPL hazır
- Modül sistemi hazır
- Makrolar hazır
- Lisp parser yazmana gerek yok

Seçenek 2: Kendi Lisp motorunu C ile yazmak

Bu daha Emacs'ın eski çekirdeğine benzer.

Şunları yazarsın:

Lexer
Parser
AST
Evaluator
Environment
Garbage Collector
Macro System

Örnek:

(+ 1 2)

önce:

(
+
1
2
)

tokenlarına ayrılır.

Sonra:

```
List {
   Symbol("+"),
   Integer(1),
   Integer(2)
}
```

oluşturulur.

Sonra evaluator:

(+ 1 2)
=> 3

hesaplar.

Bu yol sana:

- Dil tasarımı özgürlüğü

- Daha küçük runtime
- Tam kontrol

sağlar.

Ama Emacs seviyesinde bir Lisp yapmak tek başına yıllarca sürebilir.

Emacs ne yapıyor?

Modern Emacs'ta:

C çekirdeği
  |
  ▼
Elisp Interpreter
  |
  ▼
.el dosyaları

yani Lisp motoru büyük ölçüde C ile yazılmıştır.

Ben senin PSD Ecosystem / UMFS hedefin için ne öneririm?

Başlangıçta:

C++ Core
  |
  ▼
Embedded Guile
  |
  ▼
.scm Scriptleri

kullan.

Sistem olgunlaştığında istersen:

C++ Core
  |
  ▼
PSD Lisp

adında kendi Lisp'ini geliştirirsin.

Bu yol hem GNU felsefesine yakın hem de geliştirme maliyetini çok düşürür. Guile aslında sana hazır bir "Emacs tarzı Lisp motoru" verir; sıfırdan interpreter yazmaya göre çok daha hızlı ilerlersin.